

Desenvolvimento de API RESTful com Django

1. Visão Geral

Desenvolver uma API RESTful robusta em Python utilizando o framework **Django** em conjunto com o **Django REST Framework (DRF)**, fundamentada em boas práticas de arquitetura de software, conformidade estrita com o protocolo HTTP e persistência em banco de dados relacional.

Cada grupo deve selecionar um domínio de negócio (ex.: *E-commerce*, *Gestão Hospitalar*, *Controle Financeiro*, *Streaming*, etc.) e modelar no mínimo **duas entidades com relacionamento explícito (1:N ou N:N)**, implementando o ciclo de vida completo (CRUD) para ambas as entidades.

2. Requisitos Obrigatórios

2.1. Modelagem e Persistência de Dados (Django ORM)

- **Banco de dados:** **SQLite** (padrão) ou **MySQL** (configurado no `settings.py` via `DATABASES`).
- **Mapeamento Objeto-Relacional (ORM):** Uso dos `models.Model` nativos do Django.
- **Migrações:** Controle rigoroso de schema utilizando o sistema nativo de migrações (`makemigrations` e `migrate`).
- **Relacionamento explícito:**
 - Configurar `models.ForeignKey` (1:N) ou `models.ManyToManyField` (N:N).

2.2. Endpoints e Métodos HTTP

A API deve contemplar operações completas para as duas entidades modeladas, cobrindo todos os verbos semânticos do HTTP:

- `GET /api/recursos/`: Listagem geral (com suporte a paginação nativa e filtros).
- `GET /api/recursos/<id>/`: Consulta detalhada de um único registro (com dados da entidade relacionada aninhados).
- `POST /api/recursos/`: Criação de um novo registro.

- `PUT /api/recursos/<id>/`: Atualização completa do registro (substituição integral).
- `PATCH /api/recursos/<id>/`: Atualização parcial de campos específicos.
- `DELETE /api/recursos/<id>/`: Remoção do registro (respeitando as regras de integridade referencial).

2.3. Status Codes e Serialização

- **Serialização e Validação:** Uso obrigatório de `ModelSerializer` para tratar validação de payload, integridade de tipos e geração/consumo de JSON.
- **Serializadores Aninhados:** Evitar ciclos de repetição e serializar o relacionamento de forma legível (usando serializers aninhados para leitura ou campos de chave primária/slug).
- **Uso semântico dos códigos HTTP:**
 - `200 OK` — Sucesso em consultas (GET) ou atualizações (PUT/PATCH).
 - `201 Created` — Criação bem-sucedida de registros (POST).
 - `204 No Content` — Exclusão bem-sucedida sem corpo de resposta (DELETE).
 - `400 Bad Request` — Requisições com parâmetros ou payload inválidos/rejeitados pelos serializers.
 - `404 Not Found` — Identificador de recurso inexistente.
 - `500 Internal Server Error` — Erros não previstos da aplicação.

2.4. Arquitetura e Estrutura do Projeto

- **Divisão em Apps:** Organização do projeto em ao menos um aplicativo modular (`python manage.py startapp core` ou o nome do domínio).
- **Camada de Visão (Views):** Os alunos podem optar por:
 - `ModelViewSet` com roteadores automáticos (`DefaultRouter`), demonstrando alta produtividade do DRF; **OU**

- `GenericAPIView` / `APIView`, demonstrando o controle granular de cada verbo HTTP.
- **Segurança e Variáveis de Ambiente:** Uso de arquivos `.env` (`python-dotenv` ou `django-environ`) para isolar a `SECRET_KEY`, status de `DEBUG` e credenciais de banco de dados.

3. Instruções de Entrega

- **Repositório:** Criar um repositório no **GitHub** contendo:
 - Todo o código-fonte do projeto (`manage.py`, `apps`, etc.).
 - Arquivo `.gitignore` configurado adequadamente (não enviar banco SQLite local nem pastas `.venv/` e `__pycache__`).
 - Arquivo de dependências (`requirements.txt`).
 - Arquivo `README.md` detalhando como configurar o ambiente virtual, rodar as migrações e iniciar o servidor (`runserver`).
- **Submissão:** Enviar o link do repositório via **BlackBoard** na atividade correspondente (**3ª Avaliação A2**).
- **Prazo Limite:** Até às 19h00 do dia da defesa do código.
- **Atenção:** *Commits realizados após as 19h00 da data limite não serão considerados para a avaliação.*

4. Defesa de Código

A avaliação contará com uma arguição técnica individual/em grupo durante a aula:

- **Demonstração prática:** Execução do servidor de desenvolvimento e teste de todos os endpoints via cliente HTTP.
- **Explicação técnica:** Perguntas pontuais sobre a modelagem no Django ORM, resolução de chaves estrangeiras, serializadores aninhados, funcionamento do roteamento e tratamento de validações.
- **Critério individual:** A nota de cada integrante considerará o domínio técnico demonstrado sobre a solução implementada.